

Magnafic AI Developer Project Document

Developer handoff and architecture reference

Contents

1. Project Overview
2. Repository Structure
3. Technology Stack
4. High-Level Architecture
5. Applications
6. Backend Module Map
7. Database Model Summary
8. Main Workflows
9. Environment Configuration
10. Local Development
11. Build, Test, and Validation
12. Deployment Model
13. Extension Guidelines
14. Existing Documentation
15. Current Project Status
16. Quick Developer Orientation

This document is for developers joining or reviewing the Magnafic AI codebase. It explains what has been built, how the system is organized, how the main workflows move through the application, and where future developers should look when extending the platform.

1. Project Overview

Magnafic AI is an AI-assisted consulting and business intelligence platform. The product helps teams create projects, onboard company context, discover company information, ingest knowledge documents, run research, chat with AI workforce agents, and generate reports.

The system is implemented as a pnpm monorepo with a modular monolith backend, a Next.js frontend, Prisma/PostgreSQL persistence, Redis-backed queues, LiteLLM model access, pgvector-based retrieval, and an optional Sanity Studio for managing workforce agent profiles.

The current platform is developer-oriented around these capabilities:

- Authenticated project workspace.
- Project onboarding and lifecycle tracking.
- Company discovery from profile and website context.
- Company profile review and approval.
- Knowledge document upload, processing, chunking, embedding, and retrieval.
- Research plans, conversations, reports, and AI executions.
- Business-facing workforce agents backed by a shared LangGraph-style runtime.
- Admin/platform visibility for users, model providers, prompts, billing, audit logs, health, agent runs, and tool executions.

2. Repository Structure

```
.
|-- apps/
|   |-- api/           NestJS API, modules, guards, workers, runtime services
|   |-- web/           Next.js app router frontend
|   `-- magnafic-ai/   Sanity Studio for workforce agent content
|-- packages/
|   |-- contracts/     Shared DTO/contracts and API-facing types
|   |-- domain/        Shared domain primitives and project lifecycle helpers
|   `-- ui/            Shared UI package placeholder/docs
|-- prisma/            Prisma schema, migrations, and live seed script
|-- docs/              Architecture, deployment, milestone, and handoff docs
|-- services/          Service-level notes for future worker/runtime boundaries
|-- scripts/           Workspace scripts, including Prisma client generation
|-- storage/           Local development knowledge storage
|-- tests/             Shared test area
`-- docker-compose.yml Local Postgres/Redis infrastructure
```

The workspace is defined in pnpm-workspace.yaml:

```
apps/*
packages/*
services/*
```

3. Technology Stack

Frontend:

- Next.js 16 with App Router.

- React 19.
- TypeScript.
- React Query for API state.
- Firebase client SDK for authentication.
- Zod, React Hook Form, Zustand, Framer Motion, and lucide-react.

Backend:

- NestJS 11.
- Prisma 7 with PostgreSQL.
- pgvector for document embeddings.
- Redis and BullMQ for background processing.
- Firebase Admin for identity verification.
- JWT-based platform sessions.
- LiteLLM-compatible AI gateway.
- LangGraph package for agent-runtime direction.

Content and configuration:

- Sanity Studio in apps/magnafic-ai.
- workforceAgent Sanity schema for configurable business agents.
- Database fallback for workforce agent profiles.

Testing and quality:

- Vitest for backend/frontend unit tests.
- Playwright for web E2E tests.
- ESLint and TypeScript checks.
- pnpm deploy:check runs typecheck, lint, tests, and production app builds.

4. High-Level Architecture

The platform is a modular monolith. Backend modules live in one NestJS application, but each module is expected to keep clear boundaries:

- Controllers handle HTTP only.
- Use cases/services coordinate business behavior.
- Ports define persistence or external boundaries.
- Prisma repositories implement database access.
- Shared contracts belong in packages/contracts.
- Shared domain concepts belong in packages/domain.

Primary flow:

Authentication

- > Projects
- > Project Onboarding
- > Company Discovery
- > Company Profile
- > Knowledge Base
- > Research / Conversations / Agents

-> Reports

The API module list is wired in `apps/api/src/app.module.ts`. Global guards enforce JWT authentication and role checks through `JwtAuthGuard` and `RolesGuard`.

5. Applications

5.1 Web App

Location:

`apps/web`

Important routes:

- `/login`
- `/callback`
- `/dashboard`
- `/projects`
- `/projects/:projectId`
- `/projects/:projectId/onboarding`
- `/projects/:projectId/discovery`
- `/projects/:projectId/company-profile`
- `/projects/:projectId/knowledge`
- `/projects/:projectId/research`
- `/projects/:projectId/chat`
- `/projects/:projectId/reports`
- `/projects/:projectId/reports/:reportId`
- `/projects/:projectId/settings`
- `/workforce`
- `/workforce/:agentSlug`
- `/prompt-library`
- `/model-management`
- `/billing`
- `/settings`
- `/admin`
- `/admin/users`
- `/admin/system`
- `/admin/audit-logs`

Main frontend folders:

- `apps/web/src/app`
- `apps/web/src/components/platform`
- `apps/web/src/features/projects`
- `apps/web/src/features/onboarding`
- `apps/web/src/features/discovery`
- `apps/web/src/features/company-profile`
- `apps/web/src/features/knowledge`
- `apps/web/src/features/research`
- `apps/web/src/features/conversations`
- `apps/web/src/features/agents`
- `apps/web/src/features/reports`
- `apps/web/src/features/platform-data`
- `apps/web/src/lib/api`
- `apps/web/src/lib/auth`
- `apps/web/src/lib/firebase`

API access is centralized under `apps/web/src/lib/api`. Authentication/session helpers live under `apps/web/src/lib/auth` and Firebase initialization lives in `apps/web/src/lib/firebase`.

5.2 API App

Location:

`apps/api`

The API is a NestJS modular monolith. Each business area is implemented under `apps/api/src/modules`.

Current modules:

- admin
- agents
- ai
- audit-logs
- auth
- billing
- company-discovery
- company-profile
- conversations
- discovery-jobs
- knowledge-base
- model-management
- notifications
- onboarding
- project-profile
- projects
- prompt-library
- reports
- research
- users
- website-analysis

Shared infrastructure:

- `common/auth`: JWT guard, roles guard, decorators, authenticated-user types.
- `common/prisma`: Prisma service/module.
- `common/queue`: BullMQ queue infrastructure.
- `common/redis`: Redis connection and health.

5.3 Sanity Studio

Location:

`apps/magnafic-ai`

Sanity stores optional workforce agent profiles. The API attempts to read enabled Sanity agents first when Sanity is configured, then falls back to database-seeded `BusinessAgentProfile` records.

The main schema is:

`apps/magnafic-ai/schemaTypes/workforceAgent.ts`

Seed scripts:

`apps/magnafic-ai/scripts/seed-workforce-agents.mjs`
`apps/magnafic-ai/scripts/seed-workforce-agents.cli.mjs`

6. Backend Module Map

Auth

Responsible for:

- Verifying Firebase identity.
- Creating platform sessions.
- Returning the current user.
- Logging out.

Primary endpoints:

```
POST /v1/auth/session
GET /v1/auth/me
POST /v1/auth/logout
```

Key files:

```
apps/api/src/modules/auth
apps/api/src/common/auth
```

Projects

Responsible for project CRUD, project ownership, status, lifecycle state, and slug uniqueness per creator.

Primary endpoints:

```
POST /v1/projects
GET /v1/projects
GET /v1/projects/:projectId
PATCH /v1/projects/:projectId
DELETE /v1/projects/:projectId
```

Project Profile and Onboarding

Project profile captures company and business context required before discovery and AI readiness. Onboarding transitions the project lifecycle through the early setup states.

Primary endpoints:

```
GET /v1/projects/:projectId/profile
PUT /v1/projects/:projectId/profile
POST /v1/projects/:projectId/onboarding/start
POST /v1/projects/:projectId/onboarding/complete
```

Important lifecycle states:

```
CREATED
ONBOARDING
DISCOVERY_PENDING
DISCOVERY_RUNNING
DISCOVERY_COMPLETED
KNOWLEDGE_READY
AI_READY
FAILED
```

Discovery Jobs and Company Discovery

Discovery is designed to run through BullMQ. It validates website/profile inputs, extracts website content, navigation, metadata, technologies, products/services, and generates company profile outputs.

Primary endpoints:

```
GET /v1/projects/:projectId/discovery/status
```

POST /v1/projects/:projectId/discovery/retry

Key services:

apps/api/src/modules/company-discovery/application/services/website-validation.service.ts
apps/api/src/modules/company-discovery/application/services/content-extraction.service.ts
apps/api/src/modules/company-discovery/application/services/metadata-extraction.service.ts
apps/api/src/modules/company-discovery/application/services/navigation-extraction.service.ts
apps/api/src/modules/company-discovery/application/services/technology-detection.service.ts
apps/api/src/modules/company-discovery/application/services/product-service-extraction.service.ts
apps/api/src/modules/company-discovery/application/services/company-profile-generation.service.ts
apps/api/src/modules/company-discovery/application/services/discovery-orchestrator.service.ts
apps/api/src/modules/company-discovery/application/services/discovery-worker-processor.service.ts

Company Profile

Stores generated company intelligence and allows review, editing, and approval.

Primary endpoints:

GET /v1/projects/:projectId/company-profile
PATCH /v1/projects/:projectId/company-profile/:profileId
POST /v1/projects/:projectId/company-profile/:profileId/approve

Associated data:

- Company mission, vision, industry.
- Target customers.
- Products and services.
- Pain points.
- Unique selling proposition.
- Technologies.
- Competitors.
- Goals.
- Source metadata.

Knowledge Base and RAG

Handles project knowledge sources, document upload, document processing, chunking, embeddings, vector search, and RAG context retrieval.

Primary endpoints:

GET /v1/projects/:projectId/knowledge
POST /v1/projects/:projectId/knowledge
GET /v1/projects/:projectId/knowledge/documents
POST /v1/projects/:projectId/knowledge/documents
GET /v1/projects/:projectId/knowledge/documents/:documentId
POST /v1/projects/:projectId/knowledge/documents/:documentId/retry
DELETE /v1/projects/:projectId/knowledge/documents/:documentId
POST /v1/projects/:projectId/knowledge/search

Important services:

document-processing.service.ts
document-processing-worker.service.ts
document-text-extraction.service.ts
document-chunking.service.ts
embedding.service.ts
vector-search.service.ts
rag-context.service.ts
rag-config.service.ts

Local development storage defaults to:

storage/knowledge

Production should use Supabase Storage or another private object store.

Research

Responsible for research plan creation/listing/detail. Research consumes stable project and discovery boundaries rather than directly modifying discovery internals.

Primary endpoints:

```
GET /v1/projects/:projectId/research-plans
POST /v1/projects/:projectId/research-plans
GET /v1/projects/:projectId/research-plans/:researchPlanId
```

Conversations

Provides project chat threads and messages.

Primary endpoints:

```
GET /v1/projects/:projectId/conversations
POST /v1/projects/:projectId/conversations
POST /v1/conversations/:conversationId/messages
```

Agents and Agent Runtime

Agents expose business-facing workforce roles. All agents share one runtime instead of separate per-agent orchestration logic.

Business agents:

- User-visible roles such as Magnafic AI, Sales, Marketing, Finance, Legal, and Production.
- Stored as BusinessAgentProfile rows or Sanity workforceAgent documents.
- Carry instructions, capabilities, allowed specialists, allowed tools, model policy, and verification policy.

Internal specialists:

- RAG.
- Research.
- Analysis.
- Writing.
- Calculation.
- Document.

Runtime flow:

```
Load business agent profile
-> Load tenant/project/knowledge context
-> Supervisor classifies request
-> Planner creates bounded steps when needed
-> Specialist registry executes allowed specialists
-> Model router selects LiteLLM model aliases
-> Synthesis produces answer
-> Verification checks answer
-> AgentRun and AgentStep records are persisted
```

Primary endpoints:

```
GET /v1/agents
```

```
GET /v1/agents/:slug
GET /v1/projects/:projectId/agent-runs
GET /v1/agent-runs/:agentRunId
GET /v1/agent-runs/:agentRunId/status
POST /v1/projects/:projectId/agent-runs
POST /v1/projects/:projectId/agents/:slug/chat
```

Important files:

```
apps/api/src/modules/agents/application/runtime/agent-runtime.service.ts
apps/api/src/modules/agents/application/runtime/supervisor.service.ts
apps/api/src/modules/agents/application/runtime/task-planner.service.ts
apps/api/src/modules/agents/application/runtime/model-router.service.ts
apps/api/src/modules/agents/application/runtime/verification.service.ts
apps/api/src/modules/agents/application/runtime/specialists
apps/api/src/modules/agents/application/runtime/tools
```

AI

The AI module provides generic AI execution tracking and a LiteLLM-backed generation service. All provider access should go through the LiteLLM gateway, not direct provider SDKs in feature modules.

Primary endpoints:

```
GET /v1/ai/executions
POST /v1/ai/executions
POST /v1/llm/test
```

Reports

Stores project reports and report sections.

Primary endpoints:

```
GET /v1/projects/:projectId/reports
POST /v1/projects/:projectId/reports
GET /v1/reports/:reportId
```

Platform/Admin Modules

These support platform observability and administrative screens:

```
GET /v1/model-management/providers
GET /v1/prompt-library
GET /v1/billing/account
GET /v1/users
GET /v1/audit-logs
GET /v1/notifications
GET /v1/admin/health
GET /v1/admin/agent-runs
GET /v1/admin/tool-executions
```

7. Database Model Summary

The database is defined in:

prisma/schema.prisma

PostgreSQL extensions:

```
pgcrypto
pgvector
```

Core models:

- User: Firebase-linked platform user.
- Project: Main workspace object.

- ProjectProfile: Onboarding/company input for a project.
- DiscoveryJob: Discovery state, progress, attempts, and step metadata.
- CompanyProfile: Generated/reviewed company profile.
- CompanyTechnology, CompanyCompetitor, CompanyGoal: Structured company intelligence.
- ResearchSource: Stable source boundary consumed by research.
- ResearchPlan, ResearchFinding, Citation, ValidationResult: Research workflow records.
- Conversation, ConversationMessage: Project chat data.
- KnowledgeDocument, DocumentChunk: Uploaded documents, extracted text, chunks, embeddings.
- ModelProvider, ModelConfiguration: Model registry.
- AiExecution: AI execution log.
- BusinessAgentProfile: Configurable workforce agent profile.
- AgentRun, AgentStep, ToolExecution: Agent runtime traceability.
- PromptTemplate, PromptVersion: Prompt library.
- Report, ReportSection: Report artifacts.
- BillingAccount, UsageRecord: Billing and usage.
- Notification: User notifications.
- AuditLog: Security/audit events.

Most models include:

- createdAt
- updatedAt
- deletedAt
- createdBy
- updatedBy

The codebase uses soft-delete style fields broadly; new repository code should respect deletedAt filtering unless there is a deliberate administrative reason not to.

8. Main Workflows

8.1 Login and Session

1. User signs in through Firebase on the web app.
2. Web app sends Firebase identity to the API.
3. API verifies the identity through Firebase Admin.
4. API creates or resolves platform user/session.
5. Web app uses the platform API token for protected API requests.

Development auth bypass exists for local/E2E use. It must stay disabled in production.

8.2 Project Creation to AI Ready

1. User creates a project.

2. User completes project profile/onboarding.
3. Project lifecycle moves through onboarding and discovery states.
4. Discovery job extracts company/website context.
5. Generated company profile is reviewed and approved.
6. Knowledge records and research sources become available.
7. Project reaches AI-ready workflow paths for chat, agents, research, and reports.

8.3 Knowledge Document Processing

1. Document is uploaded to project knowledge.
2. File is stored locally in development or object storage in production.
3. Document record is created with processing status.
4. Worker extracts text.
5. Text is chunked.
6. Embeddings are generated.
7. Chunks are stored with pgvector embeddings.
8. RAG services retrieve relevant chunks for agent/chat context.

8.4 Workforce Agent Chat

1. User selects an agent from /workforce or a project agent surface.
2. API loads the matching BusinessAgentProfile from Sanity or database.
3. Runtime loads project and knowledge context.
4. Supervisor decides what capabilities are needed.
5. Planner creates steps for complex requests.
6. Allowed specialists run.
7. Model router resolves a LiteLLM model.
8. Final response is synthesized and verified.
9. Conversation message and agent run details are persisted.

8.5 Discovery to Research

Discovery does not directly mutate research internals. Instead, discovery outputs are stored as structured company/profile records and ResearchSource records. Research consumes those stable records, keeping discovery and research independently maintainable.

9. Environment Configuration

Example configuration is in:

```
.env.example
```

Important backend variables:

```
PORT
```

DATABASE_URL
REDIS_URL
JWT_SECRET
JWT_EXPIRES_IN_SECONDS
CORS_ORIGIN
FIREBASE_PROJECT_ID
FIREBASE_CLIENT_EMAIL
FIREBASE_PRIVATE_KEY
FIREBASE_CHECK_REVOKED
LITELLM_BASE_URL
LITELLM_API_KEY
LITELLM_DEFAULT_MODEL
SANITY_PROJECT_ID
SANITY_DATASET
SANITY_API_TOKEN
DISCOVERY_WORKER_ENABLED
DOCUMENT_WORKER_ENABLED
AGENT_WORKER_ENABLED
AUTH_BYPASS_ENABLED
AUTH_BYPASS_TOKEN

Important frontend variables:

NEXT_PUBLIC_API_URL
NEXT_PUBLIC_FIREBASE_API_KEY
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
NEXT_PUBLIC_FIREBASE_PROJECT_ID
NEXT_PUBLIC_FIREBASE_APP_ID
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
NEXT_PUBLIC_AUTH_BYPASS_ENABLED
NEXT_PUBLIC_AUTH_BYPASS_TOKEN

RAG variables:

KNOWLEDGE_STORAGE_PROVIDER
KNOWLEDGE_STORAGE_DIR
KNOWLEDGE_MAX_UPLOAD_BYTES
EMBEDDING_PROVIDER
EMBEDDING_MODEL
EMBEDDING_DIMENSIONS
RAG_CHUNK_SIZE
RAG_CHUNK_OVERLAP
RAG_MAX_RETRIEVED_CHUNKS
RAG_MAX_CONTEXT_CHARACTERS
RAG_MIN_SIMILARITY

Agent runtime variables:

MAX_AGENT_STEPS
MAX_PLANNER_STEPS
MAX_VERIFICATION_RETRIES
MAX_CONTEXT_CHUNKS
MAX_OUTPUT_TOKENS
MAX_SPECIALIST_CONCURRENCY
MODEL_POLICY_GENERAL
MODEL_POLICY_REASONING
MODEL_POLICY_RESEARCH
MODEL_POLICY_WRITING
MODEL_POLICY_VERIFICATION

10. Local Development

Install dependencies:

```
pnpm install
```

Start local infrastructure if using Docker:

```
docker compose up -d
```

Generate Prisma client:

```
pnpm prisma:generate
```

Apply local schema changes:

```
pnpm exec prisma migrate dev
```

Or sync without creating a migration in early local development:

```
pnpm exec prisma db push
```

Seed live/local admin data after setting admin env vars:

```
pnpm prisma:seed
```

Run API:

```
pnpm --filter @platform/api dev
```

Run web:

```
pnpm --filter @platform/web dev
```

Run Sanity Studio:

```
pnpm sanity:dev
```

Default local URLs:

Web: <http://127.0.0.1:3000>

API: <http://127.0.0.1:3001>

API base: <http://127.0.0.1:3001/v1>

11. Build, Test, and Validation

Root commands:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm deploy:check
```

App-specific commands:

```
pnpm --filter @platform/api typecheck
pnpm --filter @platform/api lint
pnpm --filter @platform/api test
pnpm --filter @platform/api build
```

```
pnpm --filter @platform/web typecheck
pnpm --filter @platform/web lint
pnpm --filter @platform/web test
pnpm --filter @platform/web test:e2e
pnpm --filter @platform/web build
```

Before production deployment, use:

```
pnpm deploy:check
```

12. Deployment Model

Recommended production split:

- Web: Vercel or another Next.js host.
- API/workers: Railway, Render, Fly.io, or equivalent persistent Node service.

- Database: managed PostgreSQL with pgvector.
- Redis: managed Redis.
- Storage: Supabase Storage, S3-compatible storage, or Firebase Storage.
- Sanity Studio: Sanity-hosted Studio or separately deployed Studio.

Production checklist:

- AUTH_BYPASS_ENABLED=false.
- No bypass token exposed to frontend.
- DATABASE_URL points to a production database with pgvector.
- Prisma migrations are deployed.
- Redis is reachable from API/worker process.
- Firebase Auth authorized domains include the deployed web URL.
- API CORS_ORIGIN matches the deployed web URL.
- Web NEXT_PUBLIC_API_URL points to the API /v1 URL.
- Storage uses private production credentials.
- Initial admin user is seeded.
- API docs and admin endpoints are reviewed before public exposure.

13. Extension Guidelines

When adding backend behavior:

- Add or reuse a module under apps/api/src/modules.
- Keep HTTP details in controllers.
- Put business coordination in use cases/services.
- Define ports for persistence or external dependencies.
- Implement database access in Prisma repository adapters.
- Add shared request/response types to packages/contracts when frontend and backend both need them.
- Add tests around use cases, guards, runtime services, and repository behavior with meaningful branching.

When adding frontend behavior:

- Add route files under apps/web/src/app.
- Put reusable feature UI under apps/web/src/features/<feature>/components.
- Add API calls under apps/web/src/lib/api.
- Reuse existing auth/session helpers.
- Keep project lifecycle rules aligned with packages/domain.

When adding an agent:

- Prefer adding a BusinessAgentProfile row or Sanity workforceAgent document.
- Do not create a separate orchestration implementation for each department.
- Reuse existing specialists when possible.
- Add a specialist only when a new execution capability is truly needed.
- Expose tools through controlled server-side wrappers with tenant/project validation.

When adding AI model access:

- Configure the model in LiteLLM.
- Route through the existing LiteLLM gateway.
- Use model policy variables or agent profile model policy.
- Do not put provider API keys inside LangGraph nodes, specialists, or frontend code.

When adding document/RAG behavior:

- Treat uploaded document content as untrusted.
- Preserve tenant/project scope in all retrieval calls.
- Keep extraction, chunking, embedding, and retrieval concerns separated.
- Keep embedding dimensions aligned with the configured pgvector column.

14. Existing Documentation

Useful supporting docs:

```
docs/Architecture.md
docs/Backend.md
docs/API.md
docs/Database.md
docs/AgentRuntime.md
docs/SanityWorkforceAgents.md
docs/deployment/README.md
docs/security/README.md
docs/ai/README.md
docs/architecture/*.md
docs/api/*.md
docs/database/*.md
```

This document is intended to be the single starting point. The milestone docs provide deeper historical context for individual implementation phases.

15. Current Project Status

Implemented foundation:

- Monorepo structure.
- Next.js frontend.
- NestJS backend.
- Prisma schema and migrations.
- Firebase-backed auth flow.
- Project lifecycle and onboarding.
- Company discovery architecture.
- Company profile review flow.
- Knowledge base and RAG foundation.
- LiteLLM-backed AI service.
- Shared agent runtime with business agents and specialists.
- Sanity-backed workforce agent configuration.
- Admin/platform views.

- Deployment guide and environment examples.

Areas that future developers should verify or harden before full production use:

- Production storage provider configuration.
- Background worker deployment split and operational monitoring.
- End-to-end auth/RBAC coverage for every protected route.
- Discovery worker behavior against real-world websites.
- Model/provider cost tracking and budget controls.
- Full production migration path for existing live data.
- Audit logging coverage for mutating business actions.
- API documentation exposure and security review.

16. Quick Developer Orientation

If you are new to the project, start in this order:

1. Read this document.
2. Read README.md.
3. Inspect prisma/schema.prisma.
4. Inspect apps/api/src/app.module.ts.
5. Inspect the module you need under apps/api/src/modules.
6. Inspect the matching UI feature under apps/web/src/features.
7. Check shared contracts in packages/contracts.
8. Run typecheck/tests for the package you touched.

For most feature work, the implementation path is:

```
Prisma model or existing model
-> Backend repository/use case/controller
-> Shared contract if needed
-> Frontend API client
-> Feature component/page
-> Unit or E2E coverage
```

Keep the modular boundaries intact. The project is easiest to maintain when discovery, research, knowledge, agents, and reports communicate through stable records and contracts rather than reaching into each other's internals.